

Entwicklerdokumentation

Grone DownloadPortal

Architektur, Code, Tests und Release-Prozess

ENTWICKLUNG & WARTUNG

System: ASP.NET Core 8 MVC auf IIS

URL: <https://dms-scan.grone.de/download/>

Stand: 02.09.2026

1. Zweck

Diese Entwicklerdokumentation beschreibt Aufbau, Sicherheitsmodell, Konfiguration, Tests und Release-Prozess der ASP.NET Core 8 MVC-Anwendung DownloadPortal. Ziel ist eine kleine, wartbare und ausschließlich lesende Weboberfläche für freigegebene Dateien.

2. Technologie und Projektstruktur

Bereich	Einsatz
Framework	.NET 8 / ASP.NET Core MVC
UI	Razor Views und eigene CSS-Datei
DI	Integrierter ASP.NET Core Dependency Injection Container
Konfiguration	Options Pattern mit DownloadSettings
Dateizugriff	FileDownloadService hinter IFileDownloadService
Tests	xUnit, temporäre Testverzeichnisse
Hosting	IIS, in-process, frameworkabhängige Veröffentlichung

```
DownloadPortal/  
  Controllers/HomeController.cs  
  Models/DownloadSettings.cs  
  Models/DownloadFileViewModel.cs  
  Models/DownloadIndexViewModel.cs  
  Services/IFileDownloadService.cs  
  Services/FileDownloadService.cs  
  Services/DownloadFile.cs  
  Views/Home/Index.cshtml  
  Views/Shared/_Layout.cshtml  
  wwwroot/css/site.css  
  Program.cs  
  appsettings.json  
  
DownloadPortal.Tests/  
  Services/FileDownloadServiceTests.cs
```

3. Verantwortlichkeiten der Schichten

Schicht	Verantwortung
Controller	HTTP-Eingaben prüfen, Service aufrufen, View oder Dateiantwort liefern
Service	Dateisystemzugriff, Allowlist, Pfadsicherheit, MIME-Type-Ermittlung
Models	Konfiguration, Dateimetadaten und View-Daten
Views	Darstellung; kein direkter Dateisystemzugriff
Program.cs	DI, Options-Bindung, Middleware und Routing

4. Konfigurationsmodell

```
namespace DownloadPortal.Models;

public sealed class DownloadSettings
{
    public const string SectionName = "DownloadSettings";
    public string RootPath { get; init; } = string.Empty;
    public string[] AllowedExtensions { get; init; }
        = Array.Empty<string>();
}
```

Die Konfiguration wird beim Start gebunden und validiert. Eine leere Allowlist ist ein Konfigurationsfehler, kein implizites „alles erlauben“.

5. Dependency Injection und Startkonfiguration

```
builder.Services
    .AddOptions<DownloadSettings>()
    .Bind(builder.Configuration.GetSection(
        DownloadSettings.SectionName))
    .Validate(s => !string.IsNullOrEmpty(s.RootPath),
        "DownloadSettings:RootPath muss konfiguriert sein.")
    .Validate(s => s.AllowedExtensions is { Length: > 0 },
        "Mindestens eine Dateiendung muss erlaubt sein.")
    .ValidateOnStart();

builder.Services.AddSingleton<IFileDownloadService,
    FileDownloadService>();
```

6. Sicherheitsmodell des Dateidienstes

- Nur Dateien im konfigurierten Wurzelordner werden berücksichtigt.
- `SearchOption.TopDirectoryOnly` verhindert die automatische Veröffentlichung von Unterordnern.
- Dateieendungen werden normalisiert und gegen eine case-insensitive `HashSet`-Allowlist geprüft.
- `Path.GetFileName` muss dem übergebenen Dateinamen entsprechen. Dadurch werden Pfadbestandteile abgewiesen.
- `Path.GetFullPath` und ein Root-Präfixvergleich verhindern Directory Traversal.
- MIME-Type-Erkennung erfolgt erst nach erfolgreicher Allowlist- und Pfadprüfung.
- Bei unbekanntem MIME-Type wird `application/octet-stream` verwendet.
- Das Portal implementiert keinen Upload, keine Änderung und kein Löschen.

Sicherheitsregel

Eine erkannte MIME-Type-Zuordnung bedeutet nicht, dass eine Datei freigegeben ist. Ausschlaggebend ist ausschließlich die Allowlist.

7. Servicevertrag

```
public interface IFileDownloadService
{
    IReadOnlyList<DownloadFileViewModel> GetFiles();
    DownloadFile? GetFile(string fileName);
}
```

`GetFiles` liefert nur freigegebene Dateien für die View. `GetFile` liefert bei ungültiger Eingabe, nicht erlaubter Endung, fehlender Datei oder unsicherem Pfad null.

8. Controller-Verhalten

```
[HttpGet]
public IActionResult Download(string? fileName)
{
    if (string.IsNullOrEmpty(fileName))
        return BadRequest("Es wurde kein Dateiname angegeben.");

    var file = _fileDownloadService.GetFile(fileName);
    if (file is null)
        return NotFound("Die angeforderte Datei wurde nicht gefunden.");

    return PhysicalFile(file.FullPath, file.ContentType,
        file.FileName, enableRangeProcessing: true);
}
```

Range Processing unterstützt robuste Downloads größerer Dateien. Physische Pfade werden nicht an den Client ausgegeben.

9. Views und statische Assets

- _Layout.cshtml bindet ausschließlich /css/site.css ein; JavaScript ist für die aktuelle Funktion nicht erforderlich.
- Index.cshtml erzeugt Dateikarten und verwendet MVC Tag Helper für Downloadlinks.
- System.IO.Path ist in Razor explizit zu verwenden, weil RazorPageBase bereits eine Path-Eigenschaft besitzt.
- _ViewImports.cshtml aktiviert Microsoft.AspNetCore.Mvc.TagHelpers.
- app.UseStaticFiles() muss vor dem Routing vorhanden sein.

10. Unit-Tests

Die Tests verwenden für jeden Test ein eindeutiges temporäres Verzeichnis und löschen es anschließend. Es werden keine produktiven Dateien angesprochen.

Testgruppe	Abdeckung
Positivfälle	Erlaubte Dateien, Download, Größe, Sortierung, Groß-/Kleinschreibung
Filter	Nicht erlaubte Endungen und Unterordner
Pfadsicherheit	..\", ../ und absolute Fremdpfade
Fehlerfälle	Fehlender Ordner, fehlende Datei, leerer Dateiname
Konfiguration	Leerer RootPath, leere Allowlist, Endungsnormalisierung

```
dotnet test
dotnet test --filter "GetFile_ReturnsNull_WhenExtensionIsNotAllowed"
dotnet test --collect:"XPlat Code Coverage"
```

Abnahmekriterium

Alle Tests müssen erfolgreich sein. Insbesondere darf eine nicht erlaubte TXT-Datei weder in GetFiles erscheinen noch durch GetFile ausgeliefert werden.

11. Release- und Veröffentlichungsprozess

Option	Wert
Konfiguration	Release
Zielframework	net8.0
Bereitstellungsmodus	Frameworkabhängig
Zielruntime	win-x64
Einzeldatei	Deaktiviert
ReadyToRun	Deaktiviert
Trimming	Deaktiviert
Zielordner	D:\WebApps\DownloadPortal

```
dotnet clean
dotnet test
dotnet publish -c Release -r win-x64 \
  --self-contained false \
  -o D:\WebApps\DownloadPortal
```

12. Produktive web.config

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <location path="." inheritInChildApplications="false">
    <system.webServer>
      <handlers>
        <add name="aspNetCore" path="*" verb="*"
          modules="AspNetCoreModuleV2"
          resourceType="Unspecified" />
      </handlers>
      <aspNetCore processPath=".\DownloadPortal.exe"
        stdoutLogEnabled="false"
        stdoutLogFile=".\logs\stdout"
        hostingModel="inprocess" />
    </system.webServer>
  </location>
</configuration>
```

13. Coding- und Wartungsregeln

- Namespaces projektweit konsistent als DownloadPortal.* verwenden.
- Keine Dateieindungen im Service hart codieren.
- Keine Pfade aus Querystrings direkt kombinieren oder ausliefern.
- Keine Zugangsdaten, Passwörter oder Schlüssel in appsettings.json einchecken.
- Neue Dateitypen nur nach fachlicher und sicherheitstechnischer Prüfung ergänzen.
- Bei Änderung der Dateisicherheitslogik zuerst Unit-Tests erweitern.
- UI und Service getrennt halten; keine Dateisystemlogik in Razor Views.
- Abhängigkeiten und .NET-8-Sicherheitsupdates regelmäßig prüfen.

14. Erweiterungsmöglichkeiten

- Strukturiertes Audit-Logging ohne Speicherung sensibler Inhalte.
- Ablaufdatum und Freigabestatus pro Datei über Metadaten.
- SHA-256-Prüfsumme zum Integritätsvergleich.
- Eigene Rollen oder Entra-ID-Authentifizierung, falls IIS-Standardauthentifizierung später ersetzt wird.
- Health-Check-Endpunkt nur für internes Monitoring.
- Automatisierte CI-Pipeline mit Build, Tests, Code Coverage und Release-Artefakt.

15. Definition of Done

Kriterium	Erwartung
Build	Release-Build ohne Fehler
Tests	Alle Unit-Tests grün
Security	Traversal, Fremdpfade und nicht erlaubte Typen blockiert
UI	Desktop und Mobil geprüft
Konfiguration	Produktiver RootPath und Allowlist geprüft
Deployment	Portal unter /download erreichbar
Regression	WebDAV unverändert funktionsfähig
Dokumentation	Änderungsnachweis und Betriebsdokumentation aktualisiert